pfi

Python for Fixed Income

COURSE 10 · IMPLEMENTING RAG

Serving the Retriever and What It Costs

WEEK 3 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: a service and a bill

A 10-K service and the saved runs

Service	1158 chunks, BM25 only
Answers	45 saved (day 12)
New answers	5 recorded, replayed; no key
Saved runs	266 requests, days 5 to 14
Torch	none: the server loads no model

Today

1. What a retrieval service is
2. rag_app, tested with the test client
3. A question the service does not hold
4. Why BM25 on the server
5. Taking it to a server
6. What an answer costs
7. The local route
8. The embedding cost

A service looks up and returns; it never guesses. Every cost is tokens times the price per token, read from the saved files.

How it works: a retrieval service

Why a service	A notebook serves one person; a desk uses the retriever from a form, a macro, or another program, over HTTP.
Routes	A method and a path: GET reads, POST sends a JSON body; a typed request refuses a bad body before any code of the service runs.
Checkable	A search returns each chunk with its citation; an answer returns the chunks it cites. Text with no citation is text nobody can check.
You control	The routes, what each takes, and what the service holds: the chunks, the index, the saved answers.
What comes out	One program, the same for every caller, checked by a test client in this notebook before anything is deployed.

Three routes, no model

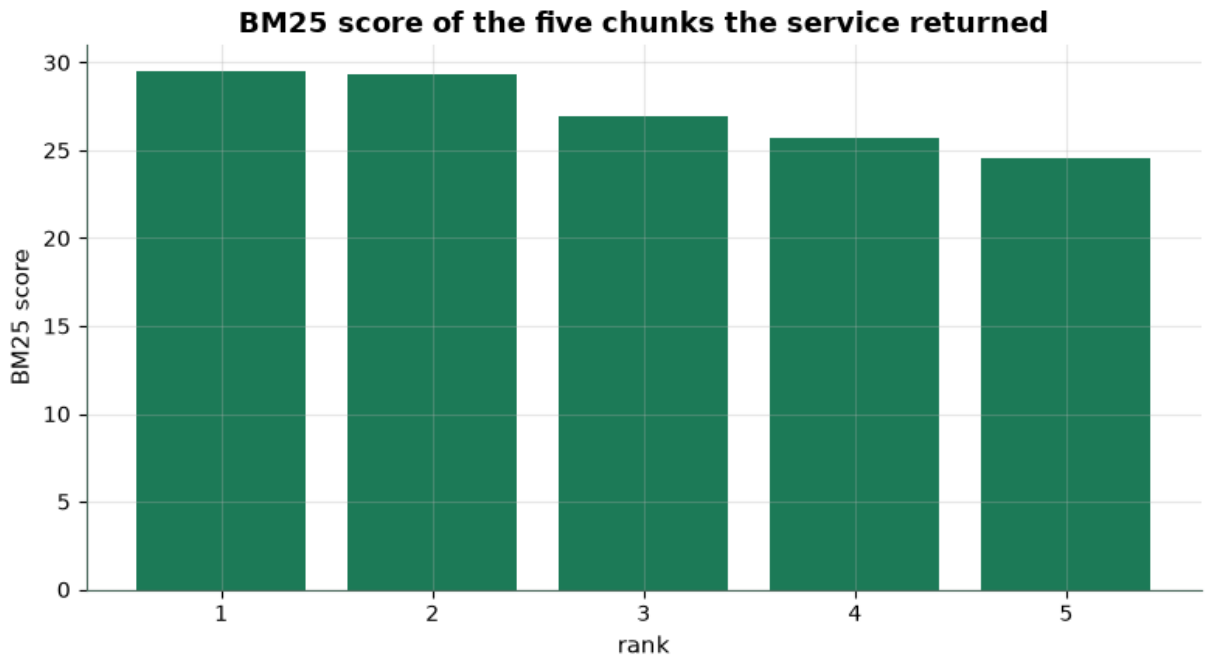
route	takes	returns	needs a model
`GET /health`	Takes: none	Returns: status, chunk count, answer count	model: no
`POST /search`	Takes: question, k (default 3), doc (optional)	Returns: the k chunks with the largest BM25 scores: chunk_id, citation, score, rank, text	model: no
`POST /answer`	Takes: question	Returns: a saved answer and the chunks it cites; the refusal line for a request for a view; a 404 otherwise	model: no

A typed request refuses a bad body before rag_app runs. Every response carries the citations a reader needs to check it.

How it works: rag_app

Three inputs	The chunk table, bm25_index(chunks), and the saved answers; two pydantic classes hold the typed requests (question, k = 3, doc).
/search	Scores every chunk with bm25_scores, drops the chunks outside doc first, returns the k largest with id, citation, score, rank, text.
/answer	refuse_first reads the question: a request for a view gets the refusal line, no lookup; else the saved answer and its cited chunks, or a 404.
No model	No route calls a model, so the same request gets the same reply on every call.
What comes out	An app object: a test client runs it in this process, a server runs it behind a port, with no change to the code.

A search through the service



POST /search, five chunks

```
GET /health: 1158 chunks, 45 answers

status 200; 5 chunks

fields: chunk_id, doc, item, page, citation, score, rank,
text

pages of the citations: 77, 39, 77, 77, 77
```

A BM25 score is a score: comparable between chunks for one question, meaningless across questions.

The answer route: saved, 404, refused

A saved question

200: the answer and the 1 chunk(s) it cites

No saved answer

404: not in the saved runs

A request for a view

200 with refused true: the refusal line, no lookup

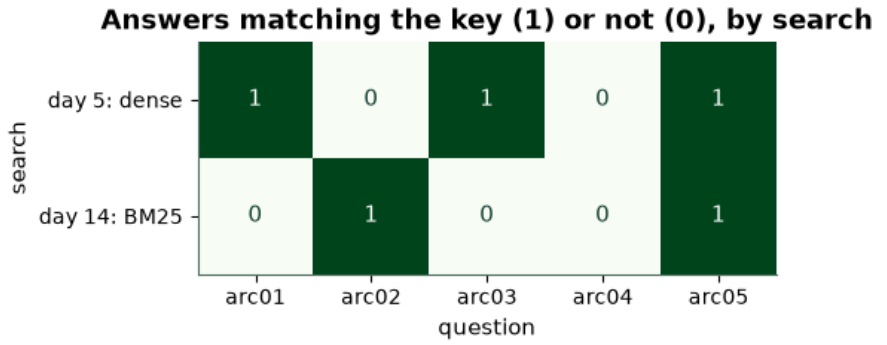
A body without a question

422: the error names "question"

```
The line returned: REFUSED: this course asks only what the document says
```

The service says what it does not hold. Generating a new answer is a request to a model, made by a client of the service.

Five arc questions through the BM25 service



question, key page, pages searched, figures, grade, check

```
arc01  39  [86, 76, 94]  no  no match  rejected

arc02  37  [38, 39, 75]  no  match  rejected

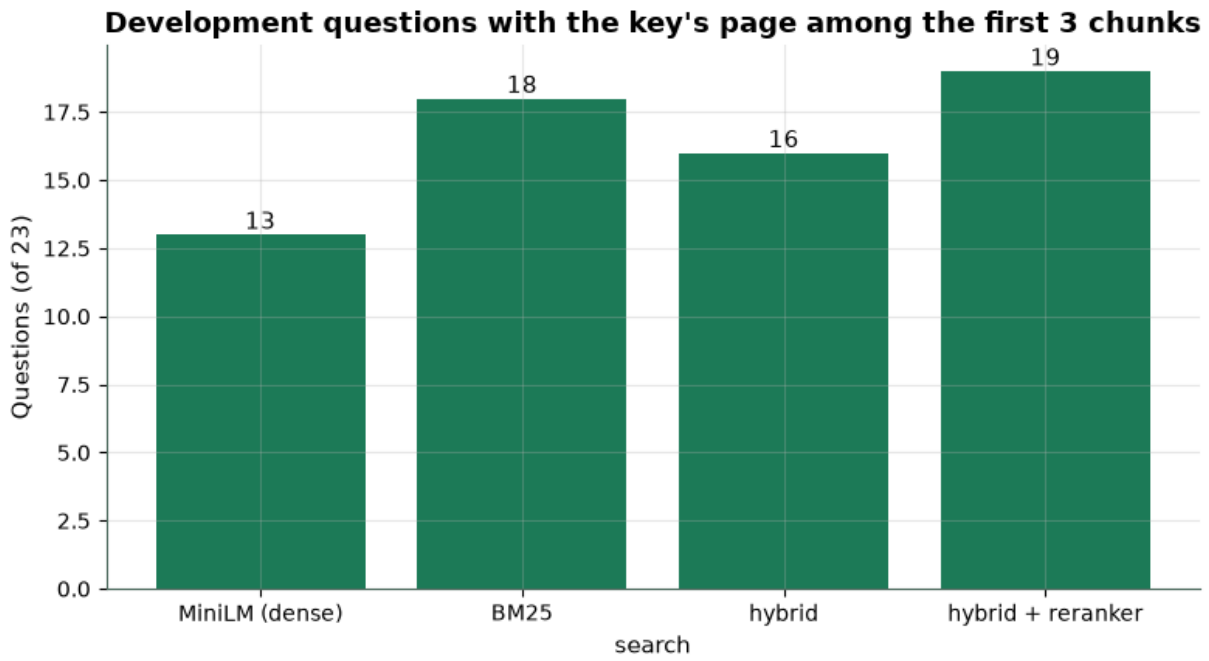
arc03  75  [75, 37, 101]  no  no match  rejected

arc04  38  [19, 38, 76]  no  no match  rejected

arc05  48  [34, 48, 74]  yes  match  rejected
```

Matching the key: day 5 dense 3 of 5, today BM25 2. Passing check_citations: 0 of 5, so none is saved: the service holds 45 answers. A match is not a citation.

What keyword search gives up



questions hit at 3 (of 23) / recall at 3

MiniLM (dense)

```
13 of 23   recall 0.5652
```

BM25

```
18 of 23   recall 0.7826
```

hybrid

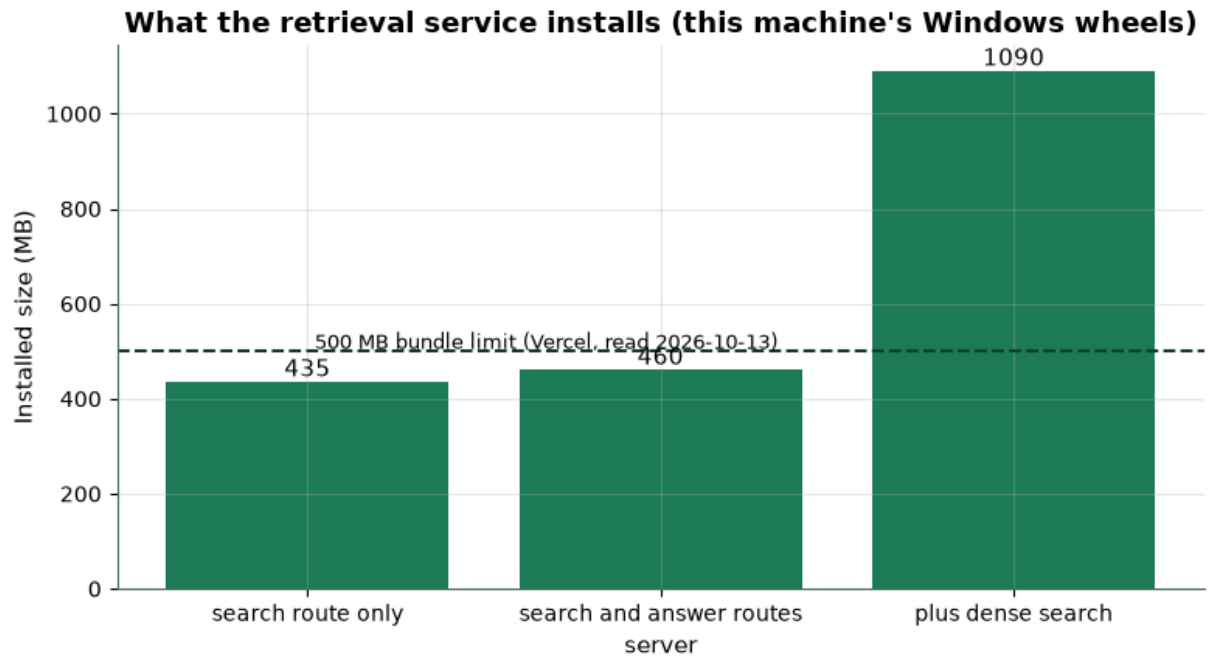
```
16 of 23   recall 0.6957
```

hybrid + reranker

```
19 of 23   recall 0.8261
```

Through the service, BM25 hit 18 of 23. A count on 23 questions about one document; development questions only.

What the server installs



server / packages / MB on disk

search route only

`65 packages  435.2 MB`

search and answer routes

`80 packages  460.2 MB`

plus dense search

`104 packages  1089.7 MB`

Vercel Python: the standard bundle limit is 500 MB uncompressed (vercel.com/docs/functions/runtimes/python, read 2026-10-13).

No PyTorch loaded, yet 54 packages: ragkit imports textkit, which imports the modules of Courses 3 and 4. Measure what a process loads.

Three facts from the platform's page

1 **A requirements file**

Vercel reads requirements.txt, pyproject.toml, or a Pipfile. requirements-server.txt: 64 pins, pywin32 left out (Windows only).

2 **An entrypoint**

app.py, index.py, server.py, main.py, wsgi.py, or asgi.py, or tool.vercel.entrypoint; the file defines a top-level app. service:app works.

3 **The bundle**

All reachable files ship; no tree-shaking for Python. The 500 MB limit is on the whole bundle: measure it on Linux first.

Vercel Python runtime: entrypoints, dependencies, and the bundle limit (vercel.com/docs/functions/runtimes/python, read 2026-10-13).

Nothing is deployed in the notebook. A platform command is never run by a notebook of this course.

How it works: what an answer costs

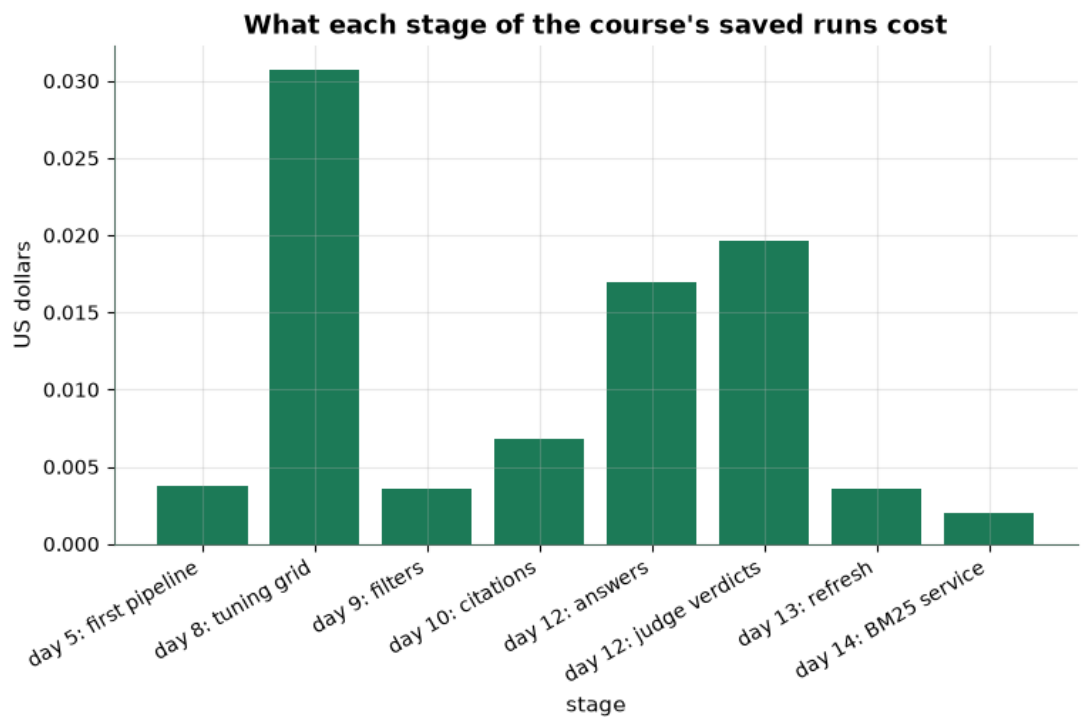
Four parts	Embedding the question, retrieving the chunks, the prompt tokens sent, the completion tokens returned; the API bills the last two.
Tokens	About four characters of English each; the prompt (system text, chunks, question) and the completion are priced separately.
cost_table	Tokens times the price per token, summed by stage, divided by the requests for the cost of one; per million, so divide by 1,000,000.
You control	The prices (a list price on its date, not a guarantee) and, for the local route, a GPU-hour price and a speed.
What comes out	One table, a row per stage: requests, tokens, dollars, dollars per request; Excel repeats it with SUMPRODUCT.

Every setting of the service's objects

Object	Setting	Default	What it controls	Today passes
rag_app	chunks	required	the chunk table the service searches; its citation column is what every result carries	the 10-K's 1,158 chunks
rag_app	bm25	required	bm25_index(chunks): the keyword index /search scores with	bm25_index(chunks)
rag_app	saved_answers	required	{question: answer text}: what /answer looks up; nothing else is ever returned	day 12's 45 answers
SearchRequest	question	required	the text to score the chunks against; a body without it is a 422	a question
SearchRequest	k	3	how many chunks /search returns, the largest BM25 scores first	5 once, 3 for the arc questions
SearchRequest	doc	None	one document to search alone, or None for all; applied before the ranking	None, boyd10k, boyd10q
AnswerRequest	question	required	the text /answer looks up, after refuse_first has read it	a saved question, an unknown one, a request for a view
cost_table	runs	required	one row per request: stage, prompt_tokens, completion_tokens	the saved runs of days 5 to 14
cost_table	prices	PRICES	US dollars per token by kind, with the date read	PRICES (the default)
cost_table	gpu_hour_usd	None	the price of a GPU hour for the local row; None adds no row	None; 1.0 in 14.7
cost_table	answers_per_hour	None	the local route's speed for the local row	None; the break-even speed in 14.7
cost_table	model	'gpt-4.1-mini'	the model whose prices are read from prices	MODEL (the default)

12 settings, 6 required, read from the signatures and the pydantic fields. A caller controls k and doc per request; the three inputs of rag_app are fixed when the service starts.

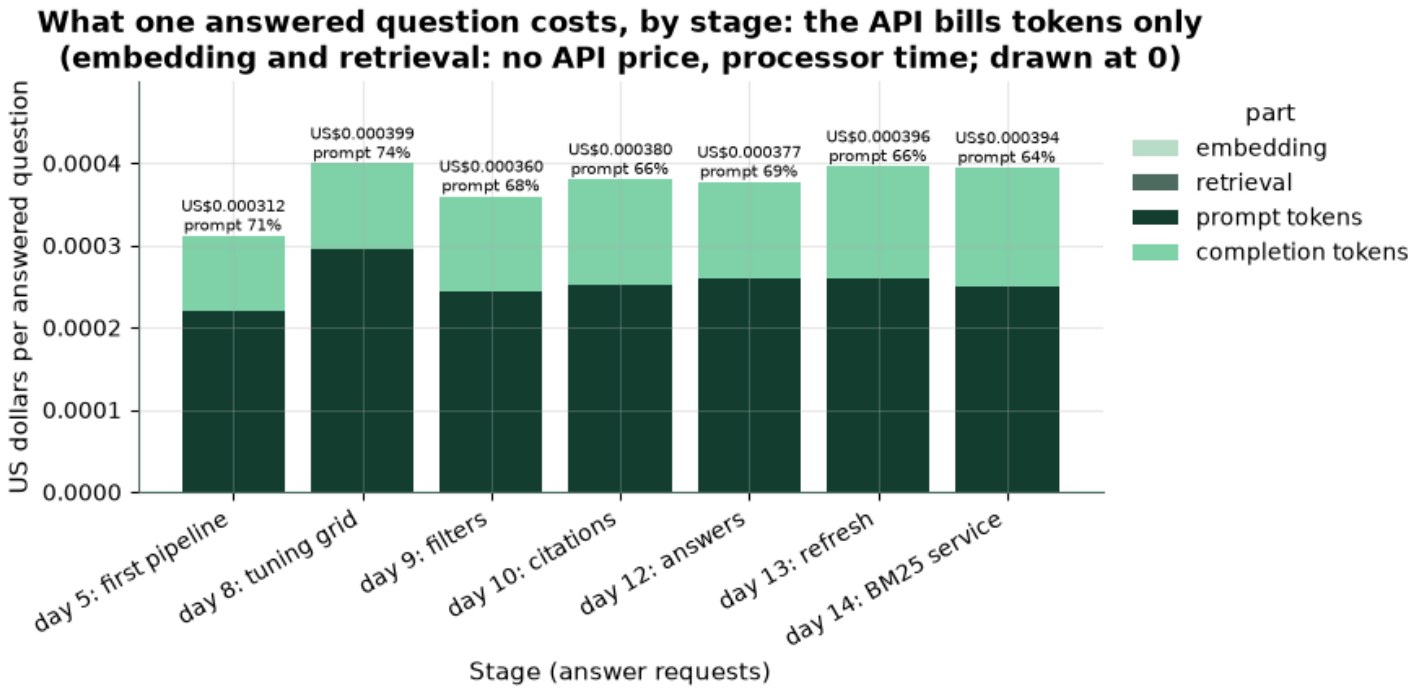
What each stage of the saved runs cost



stage	requests	US dollars
day 5: first pipeline	12	0.003738
day 8: tuning grid	77	0.030758
day 9: filters	10	0.003595
day 10: citations	18	0.006842
day 12: answers	45	0.016962
day 12: judge verdicts	90	0.019644
day 13: refresh	9	0.003562
day 14: BM25 service	5	0.001972

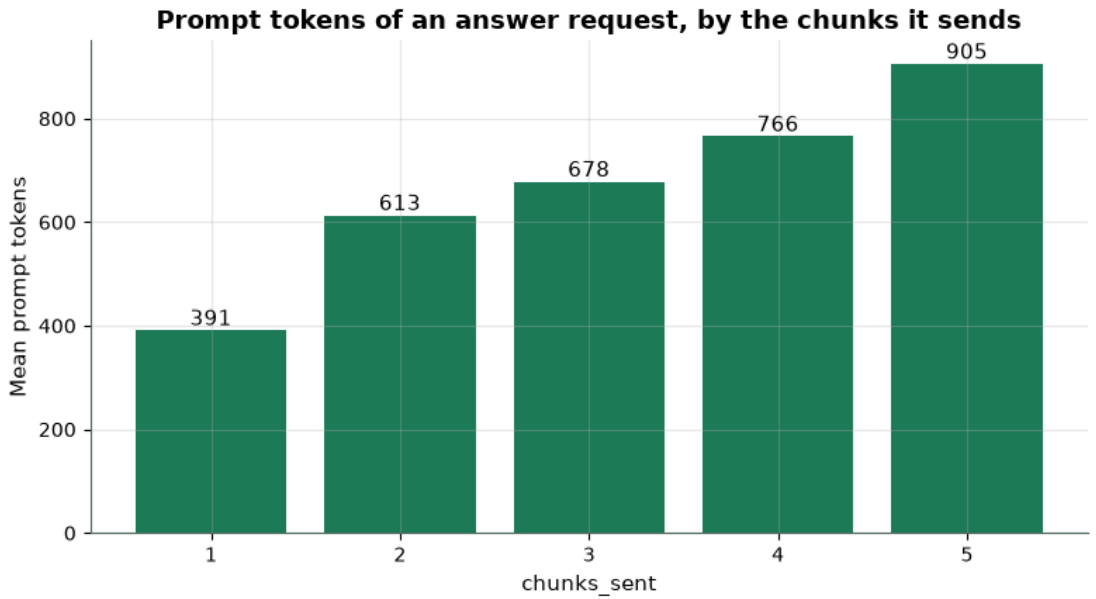
266 requests, 156,857 prompt and 15,206 completion tokens: US$0.087072 at US$0.40 and US$1.60 per million (developers.openai.com/api/docs/pricing, read 2026-10-13).

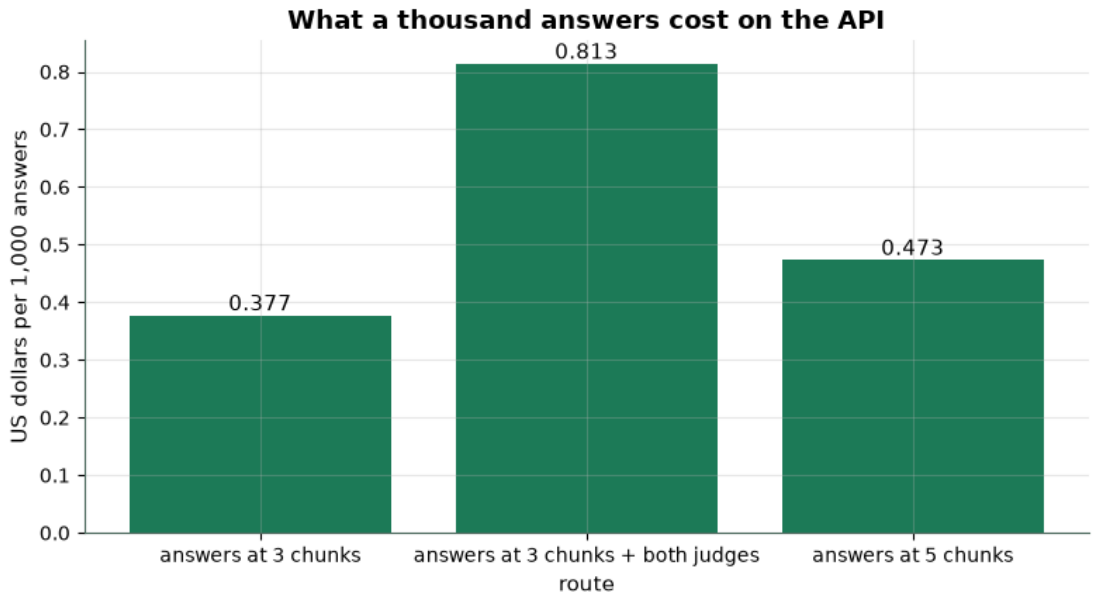
One answered question, part by part



Prompt share of an answer's cost by stage: day 5: first pipeline 71%; day 8: tuning grid 74%; day 9: filters 68%; day 10: citations 66%; day 12: answers 69%; day 13: refresh 66%; day 14: BM25 service 64%. Embedding and retrieval have no API price and are drawn at 0: the chunks sent, not the words returned, are what an answer costs.

Chunks sent, and a thousand answers





answers at 3 chunks

`US$0.3769 per 1,000 answers`

answers at 3 chunks + both judges

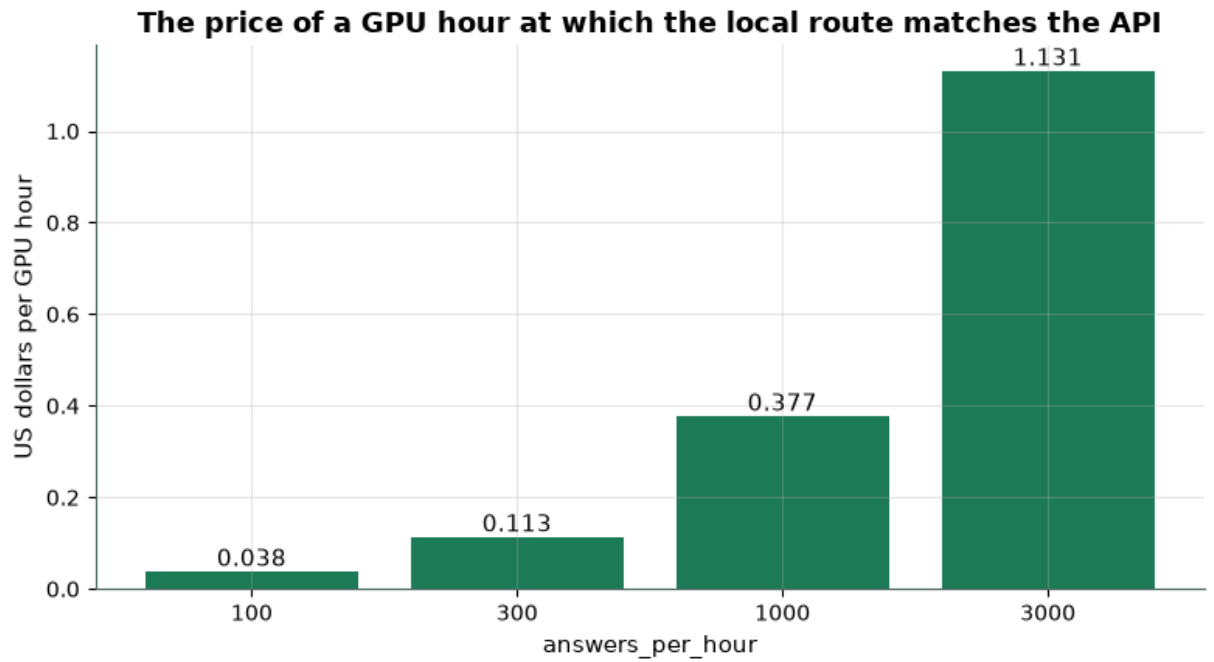
`US$0.8135 per 1,000 answers`

answers at 5 chunks

`US$0.4731 per 1,000 answers`

The prompt is nearly all of the bill, and the chunks are nearly all of the prompt. Read a real service's bill from its own logs.

A GPU hour against the API



answers an hour / break-even US$ per GPU hour

```
100 an hour    US$0.0377

300 an hour    US$0.1131

1,000 an hour    US$0.3769

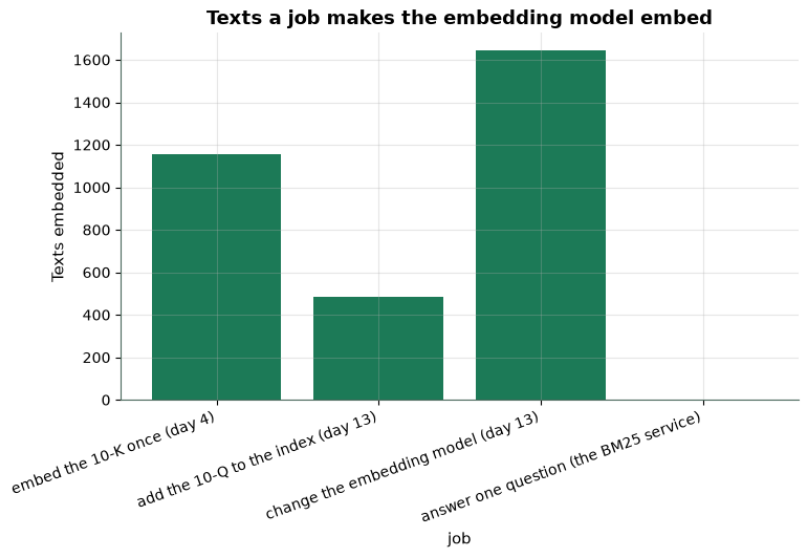
3,000 an hour    US$1.1308
```

The rates are round speeds to compute with, not measurements. Your own T4 run in Colab supplies the real one.

Colab: "Colab resources are not guaranteed and not unlimited, and usage limits sometimes fluctuate" (research.google.com/colaboratory/faq.html, read 2026-10-13).

A notebook that prints a time prints a different one every run, so this one prints none. The cell for the T4 is written and skips without a GPU.

Texts embedded, a bill in Excel, and a tag



Git: service.py, its data, and the modules in one commit (12 files); git tag week3; 65 tests.

Excel side by side (tested in Excel)

`=C2*E2+D2*F2`	equal to cost_table on all 8 rows
`=SUMPRODUCT(C2:C9,E2:E9)+SUMPRODUCT(D2:D9,F2:F9)`	0.092254; Python 0.092254
`same, prices stored as text`	0, and no error
`=AVERAGEIFS(B2:B177,A2:A177,5)`	905.3 for 5 chunks, equal to pandas

embed the 10-K once (day 4): 1158 | add the 10-Q to the index (day 13): 486 | change the embedding model (day 13): 1644 | answer one question (the BM25 service): 0

The AI check

The docstring is the prompt. The retrieval code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the prices are per million tokens and the function divides by a thousand, so every cost is 1,000 times too large.

Code that runs is not the same as code that is right.

What you can do now

- A service looks up and returns; it never guesses; its settings read from the code
- Save an answer only after the check; measure what the server loads
- Cost = tokens times the price per token (per million, divided by 1,000,000)
- Of an answer's four parts the API bills two: the prompt is the bill, and the chunks are the prompt
- Prices stored as text give a bill of zero in SUMPRODUCT

Week 3, Day 5 Course 10 capstone